

Polyglot Sandbox

Complete Study & Interview Guide

Everything from package.json to manage.sh

Concepts • Code • Interview Q&A • Cross Questions

Files Covered	Topics Inside
package.json	Dependencies, scripts, devDependencies
tsconfig.json	TypeScript compilation, target, modules
index.ts	Express server, middleware, HTTP codes
rateLimiter.ts	Redis, rate limiting, INCR, EXPIRE
hasher.ts	MD5 hashing, crypto, uniqueness
executor.ts	Docker spawn, child_process, file I/O
Python Dockerfile	Alpine, non-root user, security layers
Node.js Dockerfile	Runner isolation, WORKDIR, no CMD
API Dockerfile	Multi-stage build, docker-cli, production
docker-compose.yml	Services, volumes, networking, DooD
manage.sh	Bash automation, case statement, lifecycle

1. Project Overview

What is Polyglot Sandbox?

Polyglot Sandbox is a secure, containerized code execution engine — similar to the backend of LeetCode or Replit. When a user submits code, the system runs it safely inside an isolated Docker container and returns the output and execution time.

What It Does — In Simple Terms

- Accepts user-submitted code (Python or Node.js) via an HTTP API
- Runs it inside an isolated Docker container
- Returns the output and execution time as JSON
- Blocks abuse using Redis-based rate limiting

Key Features & WHY They Exist

Feature	Why It Exists
Docker containers	Untrusted code is dangerous — isolation prevents it from harming the server
Redis rate limiting	Stops one user from spamming requests and crashing the server
Non-root user inside container	Even if code escapes, it has no admin privileges
--network=none	Code cannot make internet requests or attack other services
--memory=256m	Infinite loops cannot eat all server RAM
MD5 + timestamp filename	Two simultaneous requests won't overwrite each other's files
spawn() not exec()	Doesn't block the server while waiting for code to finish
Multi-stage Docker build	Final API image is smaller, cleaner, no dev tools in production

Architecture — 3 Main Components

- Express API — receives requests, validates input, manages rate limiting
- Redis — ultra-fast in-memory store for tracking request counts per IP
- Docker Daemon + Runners — spawned on demand to run untrusted code safely

Complete Request Lifecycle

```
POST /execute { language: 'python', code: 'print("hi")' }
→ index.ts receives request
→ rateLimiter checks Redis → count=3 → under limit → next()
→ executeCode('python', 'print("hi")') called
→ generateHash(code + timestamp) → 'a3f8c2d1'
→ fs.writeFile → saves to /tmp/a3f8c2d1.py
→ spawn('docker', [...args]) → container created
→ Docker runs: python /code/script.py
```

```
→ stdout: 'hi' collected
→ container exits, auto-deleted (--rm)
→ duration calculated → '0.43s'
→ fs.unlink → /tmp/a3f8c2d1.py deleted
→ res.json({ output: 'hi', time: '0.43s', status: 'success' })
```

2. package.json

What Is It?

The identity card and shopping list of the project. It tells Node.js what the project is called, how to run it, and what external packages it needs. Auto-generated by running: `npm init -y`

Scripts Explained

Command	What You Type	What It Does	When To Use
build	<code>npm run build</code>	Runs <code>tsc</code> — compiles TypeScript to JavaScript into <code>/dist</code>	Before deploying
start	<code>npm start</code>	Runs the compiled JS file from <code>/dist</code>	Production
dev	<code>npm run dev</code>	Runs TypeScript directly via <code>ts-node</code> , no compile step	Development

Dependencies vs devDependencies

Package	Type	What It Does
<code>express</code>	dependency	Web framework — handles HTTP requests
<code>ioredis</code>	dependency	Redis client — lets Node.js talk to Redis
<code>typescript</code>	devDependency	The TypeScript compiler (<code>tsc</code>)
<code>ts-node</code>	devDependency	Run <code>.ts</code> files directly without compiling
<code>@types/express</code>	devDependency	Type definitions so TypeScript understands Express
<code>@types/node</code>	devDependency	Type definitions for Node.js built-ins like <code>fs</code> , <code>path</code>

Note: dependencies = needed at runtime in production. devDependencies = only during development, not deployed.

Why TypeScript Compiles to /dist

```
src/index.ts      → (tsc compiles) → dist/index.js
src/executor.ts   → (tsc compiles) → dist/executor.js
src/rateLimiter.ts → (tsc compiles) → dist/rateLimiter.js
```

TypeScript cannot run directly — it must be compiled to JavaScript first. The compiled output always has `.js` extension and lands in `/dist`.

Interview Questions

Q: Why is TypeScript used here instead of plain JavaScript?

- TypeScript adds static typing — you catch errors at compile time, not at runtime. In a security-critical project running untrusted code, a type error caught before deployment is far safer than one discovered in production.

Q: What is the difference between dependencies and devDependencies?

➤ dependencies are needed at runtime in production. devDependencies are only needed during development — they are not included when you deploy. This keeps the production build lean and secure.

Q: What does ioredis specifically do?

➤ ioredis is the connector between Node.js and Redis. Redis is a separate database. ioredis is like the phone line between your code and that database. Without it, your code has no way to read or write to Redis at all.

3. tsconfig.json

What Is It?

The instruction manual for the TypeScript compiler. When you run `npm run build` (which runs `tsc`), TypeScript reads this file to know how to compile. Auto-generated by running: `npx tsc --init`

Every Option Explained

Option	Value	What It Means
target	ES2022	Compile to ES2022 JavaScript — keeps modern syntax like <code>async/await</code>
module	CommonJS	Convert imports to <code>require()</code> style for Node.js compatibility
outDir	./dist	Put compiled .js files in /dist folder
rootDir	./src	Read .ts source files from /src folder
strict	true	Enable all strict type checking — catch errors at compile time
esModuleInterop	true	Allow clean import syntax: <code>import express from 'express'</code>
skipLibCheck	true	Don't type-check node_modules — saves time, avoids unfixable errors
forceConsistentCasing	true	Prevent filename casing bugs between Windows (case-insensitive) and Linux

The outDir + rootDir Relationship

```
rootDir = ./src    (TypeScript reads FROM here)
outDir  = ./dist   (TypeScript writes TO here)
```

```
src/index.ts      → dist/index.js
src/executor.ts   → dist/executor.js
src/rateLimiter.ts → dist/rateLimiter.js
src/hashter.ts    → dist/hashter.js
```

Interview Questions

Q: What does tsconfig.json do?

- It is the configuration file for the TypeScript compiler. It defines where source files are, where compiled output goes, and what rules to enforce during compilation.

Q: Why is module set to CommonJS instead of ESModules?

- Node.js traditionally uses CommonJS (`require`). Even though TypeScript lets us write clean import syntax, this setting converts it to `require()` at compile time for Node.js compatibility.

Q: If you add src/validator.ts — where does the compiled file appear?

- `dist/validator.js` — the extension changes from `.ts` to `.js` because TypeScript compiles to JavaScript. The directory mirrors `src` → `dist`.

4. index.ts — The Express Server

What Is It?

The front door of the entire application. Every request passes through this file. It creates the Express server, defines the /execute endpoint, and starts listening for requests.

Key Concepts

Middleware Chain

```
app.post('/execute', rateLimiter, async (req, res) => { ... })
```

Request comes in

- rateLimiter runs first
 - (if blocked → sends 429, stops here)
 - (if allowed → calls next())
- main handler runs
- response sent

express.json() Middleware

```
app.use(express.json())
```

WITHOUT it: req.body = undefined

WITH it: req.body = { language: 'python', code: 'print("hi")' }

Two Validation Layers

- Layer 1: Are language and code fields present? If not → 400 Bad Request
- Layer 2: Is the language python or nodejs? If not → 400 Bad Request
- Why return before res.status()? Without return, code continues and causes 'headers already sent' error

HTTP Status Codes — Complete Reference

Code	Name	When Used In This Project
200	OK	Default success — res.json(result) sends this automatically
400	Bad Request	Missing fields or unsupported language — user's fault
429	Too Many Requests	Rate limiter blocks — user exceeded 10 requests/min
500	Internal Server Error	catch block — something crashed on server side

401 vs 403 — Common Interview Trap

401 = I don't know who you are — please login first.

403 = I know exactly who you are — but you are not allowed here.

Interview Questions

Q: What does `app.use(express.json())` do?

- It is global middleware that parses incoming request bodies from raw JSON strings into JavaScript objects, making `req.body` accessible in all route handlers.

Cross Q: What happens if you remove it?

- `req.body` becomes undefined. Destructuring `const { language, code } = req.body` would crash immediately because you cannot destructure undefined.

Q: Why is `rateLimiter` passed as a middleware argument instead of called inside the handler?

- Express middleware runs in a chain. If `rateLimiter` blocks with 429, the main handler never runs. Calling it inside the handler would require manual return logic — messier and error-prone.

Cross Q: What does `next()` do in middleware?

- `next()` passes control to the next middleware or route handler. If `rateLimiter` calls `next()`, the request proceeds. If it sends a 429 instead, the chain stops there.

Q: Why use `async/await` here?

- `executeCode` spawns a Docker container which takes time. `async/await` lets us wait without blocking the Node.js event loop — other requests can be processed while this one waits for Docker.

5. rateLimiter.ts — Redis Rate Limiting

What is Redis?

Redis is a super-fast, in-memory key-value store. Think of it as a giant dictionary stored in RAM rather than on disk. Reading from RAM is ~100x faster than reading from a database on disk.

Key	→	Value
'ratelimit:192.168.1.1'	→	'7'
'ratelimit:203.0.113.42'	→	'3'

Why Redis in This Project?

- Every single request hits the rate limiter — needs to be extremely fast
- Works across ALL server instances — a JS object would only count per-server
- Built-in EXPIRE command auto-resets counters after 60 seconds
- INCR command is atomic — no race conditions under heavy concurrent load

The Core Logic — Step by Step

```
const currentCount = await redisClient.incr(redisKey)
// INCR: if key doesn't exist → creates at 0, increments to 1
//       if key exists → increments by 1
// 1st request → count=1, 5th → count=5, 11th → count=11

if (currentCount === 1) {
  await redisClient.expire(redisKey, WINDOW_SECONDS)
  // Only on FIRST request – starts 60 second countdown
  // After 60 seconds Redis auto-deletes the key → counter resets
}

if (currentCount > MAX_REQUESTS) {
  return res.status(429).json({ message: 'Too many requests' })
  // next() never called → main handler never runs
}
```

Timeline Example

```
0 sec → 1st request → count=1 → expire(key, 60) set
30 sec → 10th request → count=10 → allowed ■
35 sec → 11th request → count=11 → 429 blocked ■
60 sec → Redis auto-deletes key → counter gone
61 sec → next request → count=1 → fresh start ■
```

Fail Closed vs Fail Open

Strategy	What Happens if Redis Crashes	Used Here?
----------	-------------------------------	------------

Fail Open	Call next() — let requests through unprotected	No
Fail Closed	Return 500 — block all requests until Redis recovers	Yes ■

Fail closed is chosen because unprotected access would allow unlimited Docker container spawning — potentially crashing the host server.

Interview Questions

Q: How does the rate limiting work?

- Each IP's request count is stored in Redis. INCR atomically increments the counter on every request. On the first request, a 60-second expiry is set. If count exceeds 10, the request is blocked with 429. After 60 seconds, Redis deletes the key, resetting the counter.

Cross Q: Why Redis and not a JavaScript object in memory?

- A JS object only exists in one server instance. With multiple servers, each has its own counter — a user could bypass the limit by hitting different servers. Redis is centralized and shared across all instances.

Q: Why is expire only set when currentCount === 1?

- The expiry starts the 60-second window. If set on every request, the timer resets each time — a user could spam 9 requests every 59 seconds indefinitely. Setting it only on the first request creates a fixed window from that moment.

Cross Q: What does atomically mean and why does it matter?

- Atomic means the operation happens as one indivisible unit. If two requests arrive simultaneously and both read count=9, without atomicity both might get count=10 and both pass — but one should be blocked. Redis INCR guarantees each gets a unique sequential count.

6. hasher.ts — MD5 File Naming

What Is It?

A utility that converts any string into a fixed 32-character unique code. Used to generate safe, unique filenames for temporary code files saved in /tmp before Docker runs them.

The Problem It Solves

WITHOUT hashing:

User 1 submits → saves as /tmp/usercode.py

User 2 submits → saves as /tmp/usercode.py ← OVERWRITES User 1!

WITH hashing:

User 1's code → hash → /tmp/a3f8c2d1.py

User 2's code → hash → /tmp/b7e9f4a2.py ← safe, no collision

How MD5 Hashing Works

```
crypto.createHash('md5')    // create MD5 hashing machine
    .update(input)           // feed input string into it
    .digest('hex')           // output as 32-char hex string
```

Input: `"print('hello')"`

Output: `"a3f8c2d1b7e9f4a2c6d8e1f3b5a7c9d2"` (always 32 chars)

MD5 vs Cryptographic Hashing

Type	Examples	Speed	Use Case
MD5	MD5	Very fast	Non-security purposes: unique filenames, checksums
Cryptographic	SHA-256, bcrypt	Slower	Passwords, digital signatures, security

MD5 is broken for security — collisions can be crafted intentionally. But for temporary filenames, speed matters more than collision resistance.

Interview Questions

Q: Why MD5 instead of a timestamp or random number?

- A timestamp could collide if two requests arrive within the same millisecond. A random number could theoretically repeat. MD5 produces a deterministic hash based on content — same code always produces the same hash, different code always produces a different one.

Cross Q: Same code submitted by two users — same filename — is that a problem?

- Same code means same content, so even if overwritten, nothing changes functionally. The real collision risk is different code producing the same MD5 hash — astronomically rare. In `executor.ts`, a timestamp is also added to the hash input to make this even less likely.

Q: What is method chaining?

- Calling multiple methods sequentially where each returns an object.
`crypto.createHash('md5').update(input).digest('hex')` — three operations in one line instead of three separate variable assignments.

7. executor.ts — The Core Engine

What Is It?

The most important file. It takes user code, saves it to a temp file, runs it in Docker, collects the output, cleans up, and returns the result. Everything else in the project supports this file.

ExecutionResult Interface

```
interface ExecutionResult {
  output: string
  executionTime: string
  status: 'success' | 'error'    // can ONLY be one of these two
}

// TypeScript interface = contract for object shape
// If you try status: 'ok' → TypeScript error at compile time
```

Step-by-Step Execution

Step 1 — Unique Filename

```
const fileHash = generateHash(code + Date.now().toString())
// code + timestamp → always unique even for identical code

const ext = language === 'python' ? 'py' : 'js'
const filePath = path.join('/tmp', fileHash + '.' + ext)
// → '/tmp/a3f8c2d1b7e9.py'
```

Step 2 — Write File

```
await fs.writeFile(filePath, code, { encoding: 'utf-8' })
// MUST await – if Docker starts before file is written → empty file
```

Step 3 — Docker Command Built

```
docker run
  --name run_a3f8c2d1      # unique container name
  --rm                    # auto-delete when done
  --network=none           # no internet access
  --memory=256m            # max 256MB RAM
  --cpus=0.5               # max half a CPU
  -v /tmp/a3f8c2d1.py:/code/script.py:ro  # mount read-only
  polyglot-python:latest   # which image to use
  python                  # command to run
  /code/script.py          # file to execute
```

Step 4 — spawn() Runs Docker

```
const dockerProcess = spawn('docker', dockerArgs)
// spawn = runs external program non-blocking
// returns process object with event emitters
```

Step 5 — Collect Output

```
dockerProcess.stdout.on('data', (data) => { output += data.toString() })
dockerProcess.stderr.on('data', (data) => { errorOutput += data.toString() })

// stdout = normal print() output
// stderr = error messages, exceptions
// data comes as Buffer → .toString() converts to readable text
// fires multiple times as chunks arrive → concatenate all
```

Step 6 — Handle Completion

```
dockerProcess.on('close', (code) => {
  // exit code 0 = success (universal Unix convention)
  // exit code 1+ = something went wrong
  const duration = ((Date.now() - startTime) / 1000).toFixed(2) + 's'
  if (code === 0) resolve({ output, executionTime: duration, status: 'success' })
  else resolve({ output: errorOutput, executionTime: duration, status: 'error' })
})
```

Step 7 — Always Clean Up

```
} finally {
  try {
    await fs.unlink(filePath) // delete temp file
  } catch (cleanupErr) {
    console.error(...) // log but don't crash
  }
}

// finally = runs NO MATTER WHAT (success, error, or crash)
// inner try/catch = cleanup failure must not override user's response
```

spawn() vs exec() — Know This Cold

spawn()	exec()
Streams output chunk by chunk	Buffers ALL output in memory first
Non-blocking — server handles other requests	Can block if output is large
Safe for unknown/large output	Safe only for small, predictable output
Used here for Docker	Better for simple commands like git status

Interview Questions

Q: Why spawn() instead of exec()?

- spawn streams output in real time through events — it doesn't buffer everything in memory. exec waits for the process to finish and loads all output into a buffer. If user code produces large output, exec could run out of memory. spawn handles it chunk by chunk and doesn't block the event loop.

Cross Q: When would you use exec instead of spawn?

➤ `exec` is simpler and fine for short commands with predictable small output — like `git status` or `ls`. When output size is unknown or potentially large, always use `spawn`.

Q: What is the finally block and why is it critical?

➤ `finally` runs regardless of success or error. It is critical because temp files must always be cleaned up — even if Docker crashes or the Promise rejects. Without it, failed requests leave files in `/tmp`, eventually filling the disk and crashing the server.

Cross Q: Why does `finally` have its own inner `try/catch`?

➤ If `fs.unlink` fails, without the inner `try/catch`, that error would bubble up and override the actual user response — they would get a file deletion error instead of their code output. The inner `try/catch` makes cleanup failures silent — log only, don't affect the response.

Q: What does exit code 0 mean?

➤ Exit code 0 is a universal Unix convention meaning the process completed successfully. Any non-zero code means failure. Python exits with 0 if code runs without exceptions, 1 if an exception occurs. This project uses it to decide whether to return `stdout` or `stderr`.

8. Docker — Complete Understanding

What is Docker?

Docker solves the 'works on my machine' problem. It packs everything an app needs — code, runtime, dependencies — into one isolated box that runs identically everywhere.

```
Dockerfile → the recipe (instructions to build)
Image      → the cooked dish (built, ready to use)
Container  → the served plate (actually running instance)
```

What is a Container?

An isolated mini-computer running inside your real computer. It has its own filesystem, network, and resource limits but shares the host OS kernel.

```
Your Server (Host Machine)
■■■ Node.js API (running normally)
■■■ Redis (running normally)
■■■ Docker Container (isolated box)
    ■■■ Has its own filesystem
    ■■■ Has its own processes
    ■■■ Has its own memory limit
    ■■■ Runs Python completely separately
```

Container vs Virtual Machine

Container	Virtual Machine
Shares host OS kernel	Has its own complete OS
Starts in milliseconds	Starts in minutes
Megabytes in size	Gigabytes in size
Perfect for short-lived runners	Better for full isolated environments

Python Runner Dockerfile — Every Line

```
FROM python:3.12-alpine
# Download pre-built image with Python 3.12 on Alpine Linux
# Alpine = 5MB minimal OS vs Ubuntu's 180MB

RUN addgroup -S runnergroup && adduser -S runneruser -G runnergroup
# Create restricted user — no admin privileges, no home dir, no shell
# -S = system user, runs only while we tell it to

RUN mkdir /code && chown runneruser:runnergroup /code
# Create /code directory (where user code mounts)
# chown transfers ownership to runneruser
# MUST happen before USER switch — only root can chown

USER runneruser
```



```
# Switch from root to restricted user – enforces security
# All subsequent commands run as runneruser

WORKDIR /code
# Default directory – commands run from here
# No CMD – executor.ts provides the command dynamically
```

The -v Mount Flag — Critical Understanding

```
-v /tmp/a3f8c2d1.py:/code/script.py:ro

/tmp/a3f8c2d1.py  = file on HOST machine (real path)
/code/script.py   = name INSIDE container (fixed, predictable)
:ro               = read-only – container cannot modify host file
```

Why /code/script.py and not the hash name?
Because the Docker command is always: `python /code/script.py`
Fixed name = simple, predictable, always works

All Security Layers Combined

Layer	Flag/Setting	Protects Against
No internet	--network=none	Outbound requests, data exfiltration, DDoS
RAM limit	--memory=256m	Infinite loops eating server memory
CPU limit	--cpus=0.5	CPU starvation of host server
Auto-delete	--rm	Dangling containers piling up
Read-only mount	:ro	Code modifying its own source file on host
Non-root user	USER runneruser	Privilege escalation if container is escaped
Minimal OS	Alpine base	Smaller attack surface, less installed software

API Dockerfile — Multi-Stage Build

The API has its own Dockerfile to package the Express server into a container. It uses a multi-stage build to keep the final image small.

```
Stage 1 (builder):
FROM node:20-alpine AS builder
npm install (all deps including TypeScript)
npm run build → compiles TypeScript to /dist

Stage 2 (production):
FROM node:20-alpine ← fresh start, nothing from Stage 1
RUN apk add --no-cache docker-cli ← needed for spawn('docker'...)
COPY --from=builder /app/dist ./dist ← only compiled output
npm install --omit=dev ← production deps only
CMD ["node", "dist/index.js"]
```

Result:

Without multi-stage: ~500MB (TypeScript, dev deps, source files)
With multi-stage: ~150MB (only compiled JS and prod deps)

Why API Dockerfile?

Optional in development (npm run dev works fine locally). Necessary in production because: 1) Fresh servers have nothing installed, 2) docker-cli must be installed inside the container for spawn() to work, 3) Ensures identical environment everywhere.

Interview Questions

Q: What is a Docker container?

- A container is an isolated, lightweight process running on the host machine with its own filesystem, network, and resource limits. It shares the host OS kernel but is completely isolated from other processes. Think of it as a mini-computer inside your computer.

Cross Q: What is the difference between a container and a virtual machine?

- A VM emulates an entire computer including its own OS kernel — heavy, slow, gigabytes. A container shares the host OS kernel and only isolates the application layer — lightweight, starts in milliseconds, megabytes. For short-lived code runners, containers are the only practical choice.

Q: Why use Alpine instead of Ubuntu as base image?

- Alpine is a minimal Linux distribution — around 5MB vs Ubuntu's 180MB. Smaller means faster spin-up, less bandwidth, and a smaller attack surface. Every package installed is a potential vulnerability. Alpine ships with almost nothing.

Cross Q: Are there downsides to Alpine?

- Yes. Alpine uses musl libc instead of glibc. Some Python packages with C extensions can have compatibility issues on Alpine. For plain Python scripts with no special dependencies, it works perfectly.

Q: What is a multi-stage build?

- A multi-stage build uses multiple FROM instructions. Stage 1 installs all dependencies and compiles TypeScript. Stage 2 starts fresh and copies only the compiled output — leaving TypeScript, dev dependencies, and source files behind. Significantly smaller and cleaner final image.

Q: Why does the API container run as root unlike the Python/Node runners?

- The Docker socket /var/run/docker.sock is owned by root on the host. A non-root user inside the container cannot access it. Since the API needs to spawn runner containers through this socket, it must run as root. Runner containers don't need socket access so they safely use non-root.

9. docker-compose.yml

What Is It?

A tool for defining and running multi-container applications. Instead of starting Redis and the API separately with complex commands, Compose starts everything together with one command: `docker-compose up`

Two Permanent Services

```
services:
  redis:          ← always running, stores rate limit counts
  api:           ← always running, handles all HTTP requests
```

Python/Node containers are NOT here
→ they are spawned on-demand by `executor.ts`
→ deleted after each execution

Critical Volume Mounts

Mount	What It Does	Why Critical
<code>/var/run/docker.sock: /var/run/docker.sock</code>	Docker socket — lets API container talk to Docker	Without this, <code>spawn('docker...')</code> fails — API cannot create containers
<code>/tmp:/tmp</code>	Shares /tmp between host and API container	Without this, code files written by API are invisible to host

Docker-out-of-Docker (DooD)

```
HOST MACHINE
■■■ Docker Daemon (manages all containers)
■      ↑
■      talks via /var/run/docker.sock
■      ■
■■■ API Container
    ■■ Express server
    ■■ spawn('docker run...')
        → goes through socket
        → host Docker creates Python/Node container
```

Service Networking

Docker Compose creates an internal network. Services reach each other by service name as hostname. No IP addresses needed.

```
REDIS_HOST=redis
# 'redis' is the service name in docker-compose.yml
# Docker Compose makes it work as a hostname automatically
```

Without Compose: connect to '192.168.1.45:6379' ← fragile

With Compose: connect to 'redis:6379' ← always works

Interview Questions

Q: What is Docker Compose and why is it used?

- Docker Compose is a tool for defining and running multi-container apps. Instead of remembering complex Docker commands and running services in the right order manually, Compose defines everything in one file and starts it all with one command. It also handles networking between services automatically.

Cross Q: Could you run this project without Docker Compose?

- Yes but you'd manually create a network, start Redis with right settings, start the API with all environment variables and volume mounts, in the right order, every time. Compose automates and guarantees all of that.

Q: Why are both /tmp and the Docker socket mounted?

- They solve two different problems. The socket lets the API talk to Docker to create containers. The /tmp mount ensures code files written by the API container are accessible to the host's Docker daemon when mounting them into runner containers. Without /tmp sharing, files are invisible to runner containers.

Q: Is mounting the Docker socket a security risk?

- Yes — significant. Any process with access to the socket has effectively root-level access to the host because it can spawn privileged containers. In production you'd add strict API authentication, container signing, and possibly use a dedicated runtime like gVisor. For this project it's an accepted tradeoff for simplicity.

10. manage.sh — Project CLI

What Is It?

A Bash automation script — the single command centre for the entire project. Instead of remembering 10 different Docker commands, one script handles the entire project lifecycle.

All Commands

Command	What It Does
<code>./manage.sh setup</code>	Checks Docker and Git are installed, downloads base images
<code>./manage.sh build</code>	Builds all 3 Docker images, tags with git commit hash
<code>./manage.sh test</code>	Starts all services, sends test requests via curl
<code>./manage.sh clean</code>	Stops everything, removes all images and containers, clears /tmp
<code>./manage.sh logs</code>	Streams live logs, highlights ERROR and CRITICAL in red

Key Bash Concepts

set -e

```
set -e
# Exit immediately if ANY command fails
# Without it: docker build fails → script continues → confusing errors
# With it:    docker build fails → script stops → fix before continuing
```

Case Statement

```
case "$1" in
  setup) ... ;;
  build) ... ;;
  *)    echo 'Usage: ...' ;;
esac

# $1 = first argument: ./manage.sh build → $1='build'
# *) = default case, catches unrecognised commands
```

Git Commit Tagging

```
GIT_COMMIT=$(git rev-parse --short HEAD 2>/dev/null || date +%s)
docker build -t polyglot-python:latest -t polyglot-python:${GIT_COMMIT} ...

# :latest    → always points to newest build (used by docker-compose)
# :a3f8c2d   → permanent version tag (enables rollback to specific build)
```

|| true — Overriding set -e

```
docker-compose down -v || true
# If containers aren't running, 'down' fails
# || true = even if this fails, treat as success and continue
# Deliberate exception to fail-fast rule for harmless expected failures
```

Interview Questions

Q: What is `manage.sh` and why is it useful?

- It is a Bash automation script that centralises all project operations into single memorable commands. Without it, developers must remember complex Docker commands with many flags in the correct order. The script eliminates human error and makes onboarding new developers much faster.

Cross Q: Why Bash and not a Node.js script?

- Bash is available on virtually every Unix/Linux system without installing anything. A Node.js script would require Node.js to already be installed — which defeats the purpose of a setup script that runs before anything is installed.

Q: Why are images tagged with both `:latest` and a git commit hash?

- `:latest` is a moving pointer — always refers to the most recent build. The git hash is permanent — pins a specific build to exact code. This enables rollbacks: if a new build breaks something, you can redeploy the previous commit's image without rebuilding.

11. Interview Simulation — All Questions

Project Introduction Questions

Q: Tell me about this project in 2 minutes.

► Polyglot Sandbox is a containerized code execution engine — similar to the backend of LeetCode or Replit. Users submit Python or Node.js code via a REST API. The system validates input, checks rate limits using Redis, writes the code to a temp file, runs it inside an isolated Docker container with strict security constraints, collects the output, cleans up, and returns the result as JSON. The key security measures are Docker isolation with no network access, memory and CPU limits, a non-root user inside containers, and Redis-based rate limiting to prevent abuse.

Q: What were the biggest technical challenges?

► Three main challenges: 1) Running untrusted code safely — solved with Docker isolation, non-root user, and resource limits. 2) Race conditions with simultaneous requests — solved with MD5+timestamp filename hashing. 3) Not blocking the Node.js event loop while Docker runs — solved with spawn() and Promise wrapping for async handling.

Architecture Questions

Q: Why Docker for code isolation instead of just running the code directly?

► Running untrusted code directly on the server is extremely dangerous. Without isolation, malicious code could access the filesystem, make network requests, consume unlimited resources, or crash the server. Docker provides multiple isolation layers: its own filesystem, no network, strict memory and CPU limits, and a non-root user. Even if malicious code runs, it cannot escape these boundaries.

Cross Q: What other isolation approaches could you have used?

► VM-based isolation — heavier but stronger. WebAssembly sandboxing — lighter but limited language support. OS-level containers like gVisor — stronger security than Docker. For this use case Docker provides the best balance of security, speed, and simplicity.

Q: How does the system handle concurrent requests?

► Node.js handles concurrency through its event loop. spawn() is non-blocking — while one Docker container runs, the event loop continues serving other requests. Each request gets a unique filename via MD5+timestamp hash so files don't collide. Redis INCR is atomic so rate limit counts are accurate even under heavy concurrent load.

Q: What happens if a user submits an infinite loop?

► The --memory=256m flag crashes the container if it tries to cache variables infinitely. The --cpus=0.5 flag prevents it from consuming all CPU. You could also add a timeout using docker run --timeout flag or kill the container after a set duration — this would be a good improvement.

Code-Specific Questions

Q: Walk me through what happens when POST /execute is called.

➤ 1) Express receives the request and `express.json()` parses the body. 2) `rateLimiter` middleware checks Redis — if over 10 requests in 60 seconds, returns 429. 3) Input is validated — language and code must be present, language must be `python` or `nodejs`. 4) `executeCode` is called — generates unique filename with MD5+timestamp, writes code to `/tmp`. 5) `spawn` runs Docker with security flags. 6) `stdout` and `stderr` are collected via event listeners. 7) On close event, exit code determines success or error. 8) Duration is calculated. 9) finally block deletes the temp file. 10) Result is returned as JSON.

Q: Why wrap spawn in a Promise?

➤ `spawn` is event-based — it fires events like `data` and `close` rather than returning a Promise. To use `await` with it, we wrap it in `new Promise`. The Promise resolves when the `close` event fires, meaning Docker has finished. This lets us write clean `async/await` code while using an inherently event-driven API.

Q: What is Docker-out-of-Docker and why is it used here?

➤ The Express API runs inside a Docker container but needs to spawn new containers for code execution. By mounting the host's Docker socket into the API container, the API can send commands to the host's Docker daemon. This creates sibling containers on the host rather than nested containers. The `docker-cli` package is installed in the API container to provide the `docker` command.

Code Writing Questions

Q: Write a basic Express endpoint with JSON validation.

```
import express, { Request, Response } from 'express'

const app = express()
app.use(express.json())

app.post('/execute', async (req: Request, res: Response) => {
  const { language, code } = req.body

  if (!language || !code) {
    return res.status(400).json({ status: 'error', message: 'Missing fields' })
  }

  if (language !== 'python' && language !== 'nodejs') {
    return res.status(400).json({ status: 'error', message: 'Unsupported language' })
  }

  res.json({ status: 'success', language, code })
})

app.listen(3000, () => console.log('Running on port 3000'))
```

Q: Write a Redis rate limiter middleware.

```
import { Request, Response, NextFunction } from 'express'
import Redis from 'ioredis'

const redis = new Redis({ host: 'redis', port: 6379 })
const MAX_REQUESTS = 10
const WINDOW_SECONDS = 60
```



```

export const rateLimiter = async (req: Request, res: Response, next: NextFunction) => {
  const key = 'ratelimit:' + req.ip
  const count = await redis.incr(key)

  if (count === 1) await redis.expire(key, WINDOW_SECONDS)

  if (count > MAX_REQUESTS) {
    return res.status(429).json({ message: 'Too many requests' })
  }

  next()
}

```

Q: Write a function to run a shell command and capture output.

```

import { spawn } from 'child_process'

function runCommand(command: string, args: string[]): Promise {
  return new Promise((resolve) => {
    const process = spawn(command, args)
    let output = ''

    process.stdout.on('data', (data) => { output += data.toString() })
    process.on('close', (code) => { resolve(output) })
  })
}

// Usage:
const result = await runCommand('docker', ['ps'])

```

Future Improvements — Show You Think Ahead

Q: How would you improve this project?

- Several improvements come to mind: 1) Add more language runners — Java, Go, C++ — each needs its own Dockerfile. 2) Add execution timeout — kill containers that run longer than 10 seconds to prevent resource hogging. 3) LRU caching in Redis — identical code snippets return cached output instantly without Docker spin-up. 4) WebSocket connections for streaming output progressively rather than waiting for full completion. 5) CI/CD with GitHub Actions to automatically build and deploy on every git push. 6) Replace Docker socket mount with gVisor for stronger security isolation.

12. Quick Reference Cheat Sheet

HTTP Status Codes

Code	Name	Meaning
200	OK	Success
201	Created	Resource created
400	Bad Request	User sent wrong/missing data
401	Unauthorized	Not logged in
403	Forbidden	Logged in but not allowed
404	Not Found	Route doesn't exist
429	Too Many Requests	Rate limited
500	Internal Server Error	Server crashed

Docker Commands Quick Reference

Command	What It Does
<code>docker build -t name:tag -f Dockerfile .</code>	Build image from Dockerfile
<code>docker run --rm --network=none image</code>	Run container, auto-delete, no network
<code>docker ps</code>	List running containers
<code>docker images</code>	List built images
<code>docker-compose up -d</code>	Start all services in background
<code>docker-compose down</code>	Stop and remove all services
<code>docker-compose logs -f</code>	Stream live logs

Key Terms Glossary

Term	Simple Definition
Middleware	Function that runs between request and response in Express
<code>spawn()</code>	Node.js function to run external programs non-blocking
INCR	Redis command to atomically increment a counter
EXPIRE	Redis command to auto-delete a key after N seconds
Alpine	Tiny 5MB Linux OS used as Docker base image
DooD	Docker-out-of-Docker — API container controls host Docker
Multi-stage build	Dockerfile with multiple FROM — final image contains only production code
Atomic	Operation that happens as one indivisible unit — cannot be interrupted
exit code 0	Universal Unix convention for process success
<code>:ro</code>	Read-only flag on Docker volume mount
<code>next()</code>	Express function to pass control to next middleware
interface	TypeScript contract defining the shape of an object

Promise	Object representing future completion of async operation
shebang (<code>#!/bin/bash</code>)	First line of shell script — tells system which interpreter to use

File Roles Summary

File	Role
package.json	Project identity, dependencies, run scripts
tsconfig.json	TypeScript compiler configuration
src/index.ts	Express server entry point, /execute endpoint
src/rateLimiter.ts	Redis-based IP rate limiting middleware
src/hasher.ts	MD5 filename generation utility
src/executor.ts	Core engine — writes file, runs Docker, returns output
containers/python/Dockerfile	Builds secure Python code runner image
containers/nodejs/Dockerfile	Builds secure Node.js code runner image
containers/api/Dockerfile	Builds Express API server image (multi-stage)
docker-compose.yml	Defines and connects all permanent services
scripts/manage.sh	CLI automation for setup, build, test, clean, logs